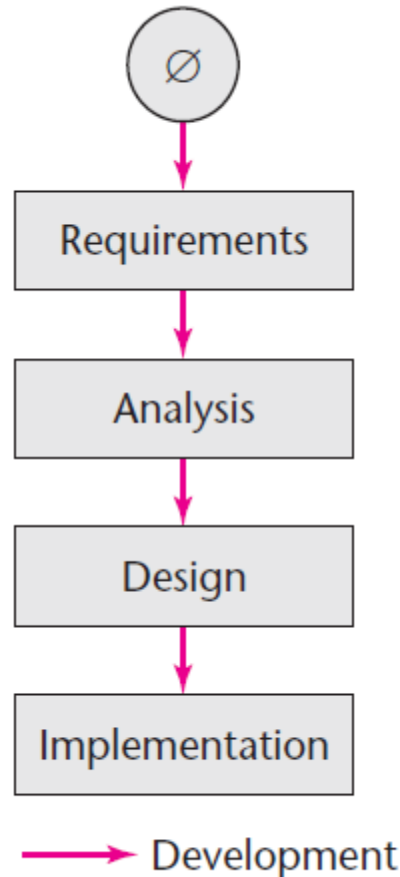


UNIT 2 RECAP

Software Life-Cycle Models

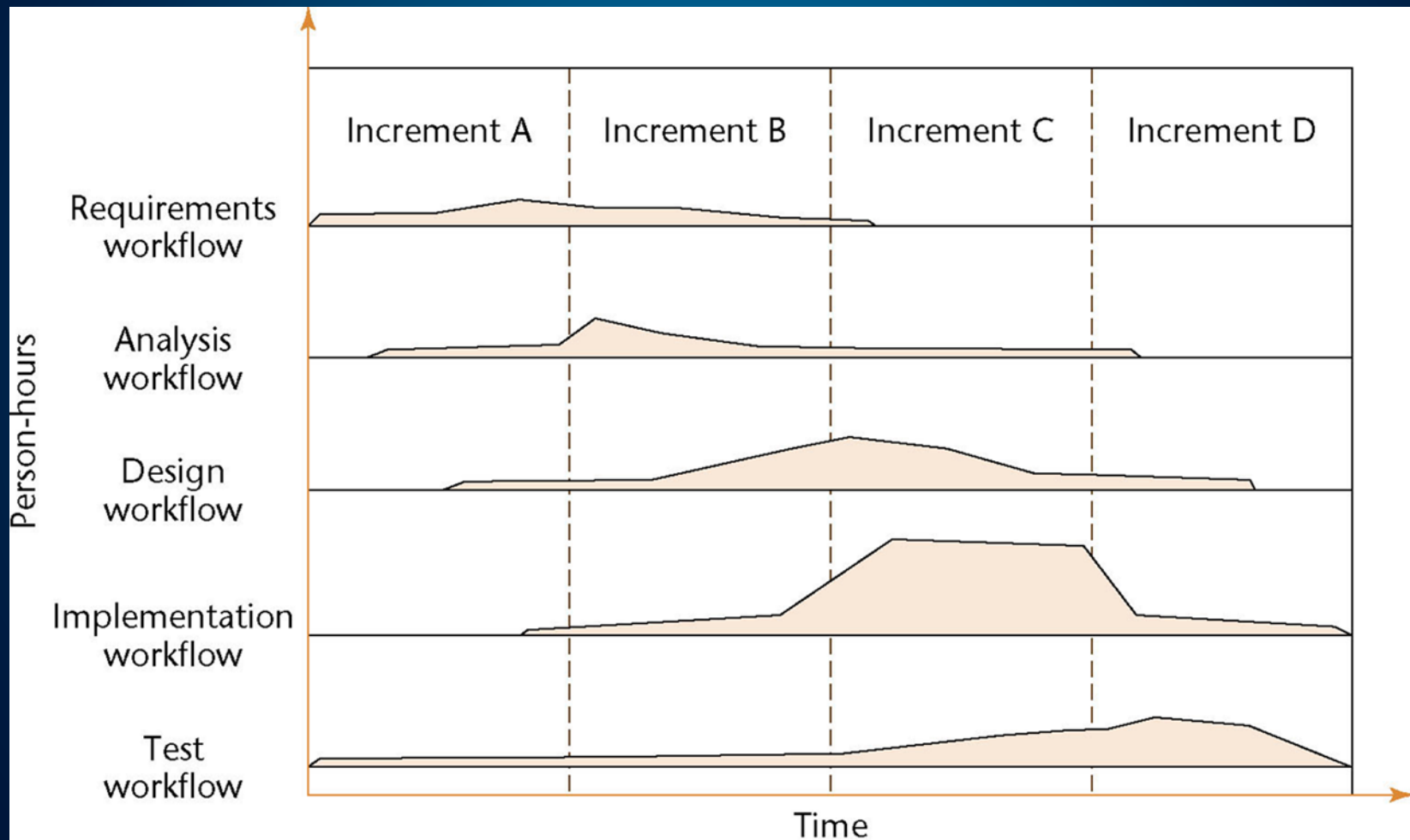
FIGURE 2.1

Idealized
software
development.



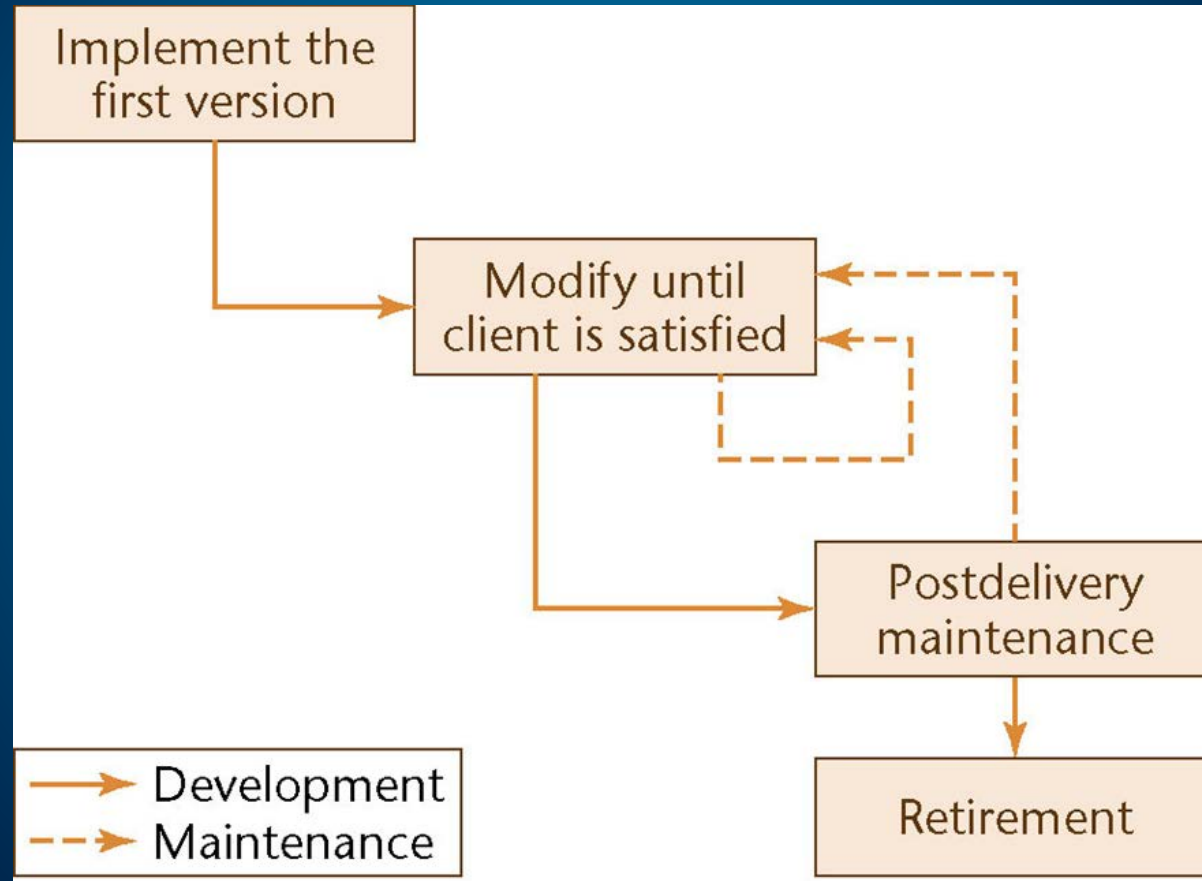
Software Life-Cycle Models

- Iteration and Incrementation Model



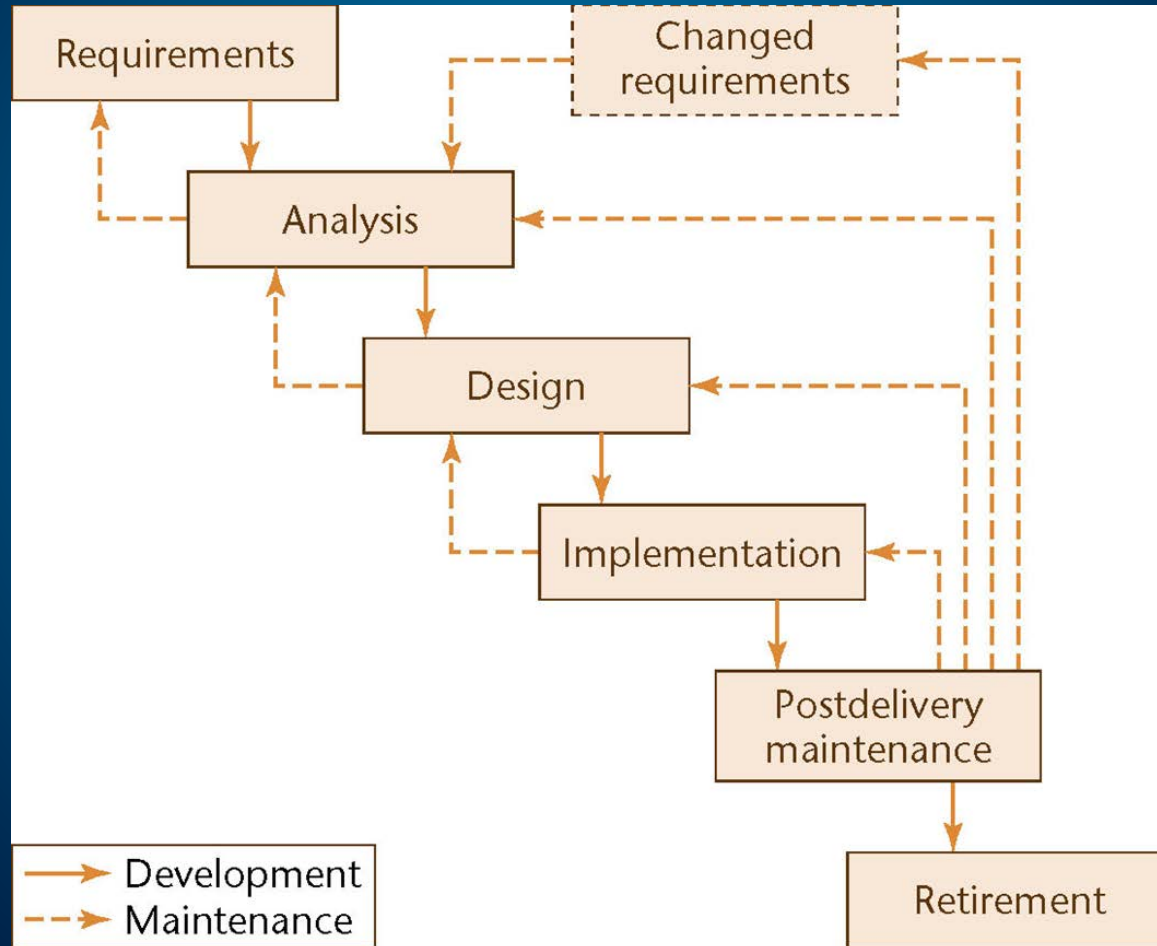
Software Life-Cycle Models

- Code-and-Fix Life-Cycle Model



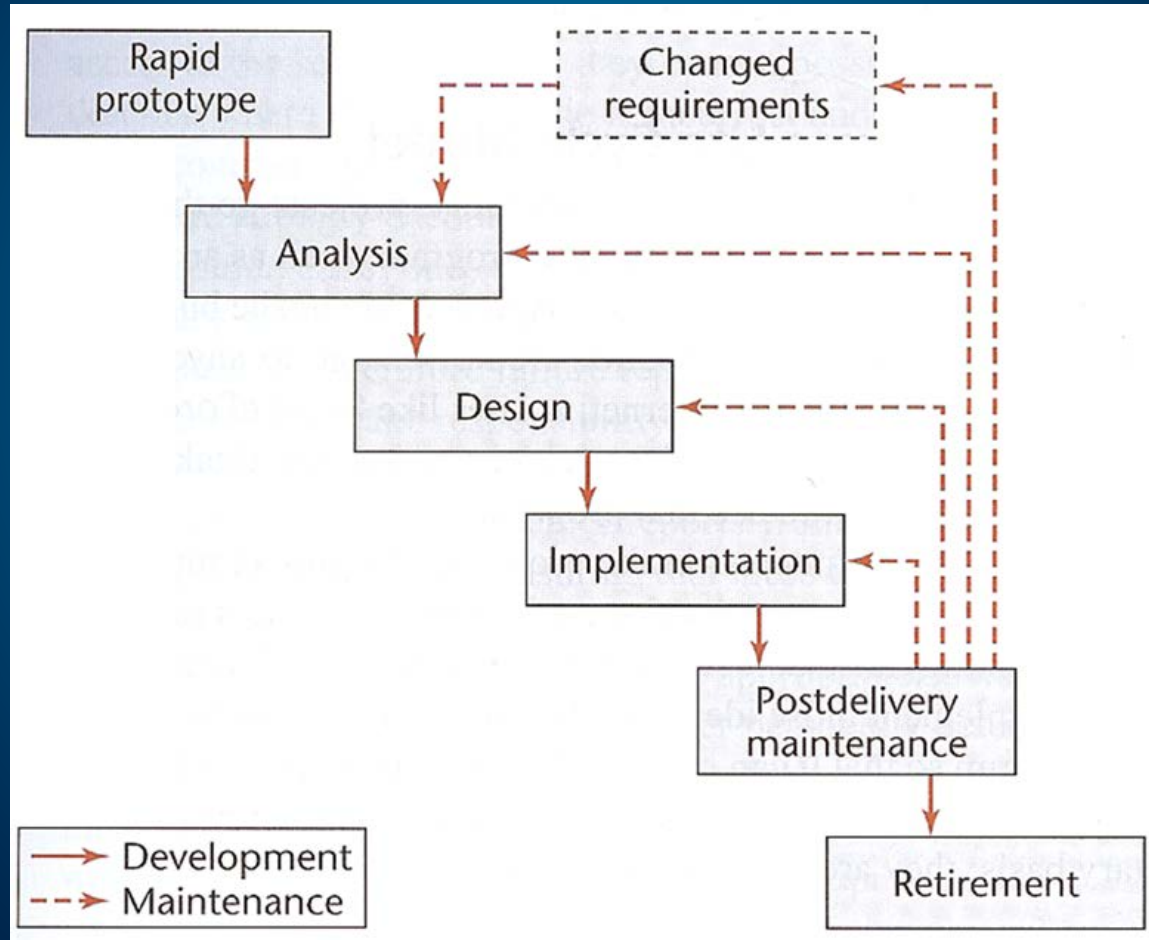
Software Life-Cycle Models

- Waterfall Life-Cycle Model



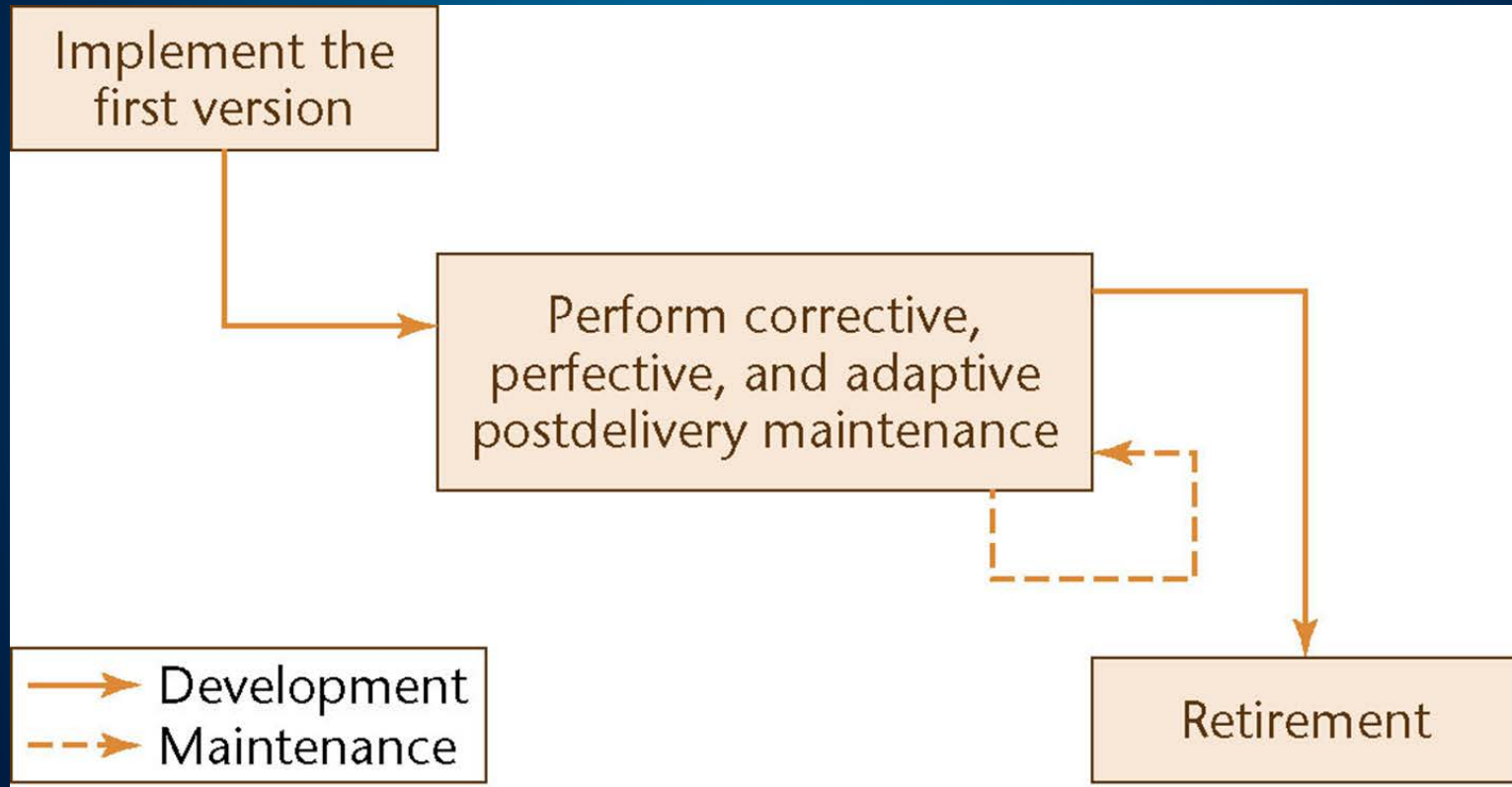
Software Life-Cycle Models

- Rapid-Prototyping Life-Cycle Model



Software Life-Cycle Models

- Open-Source Life-Cycle Model



Software Life-Cycle Models

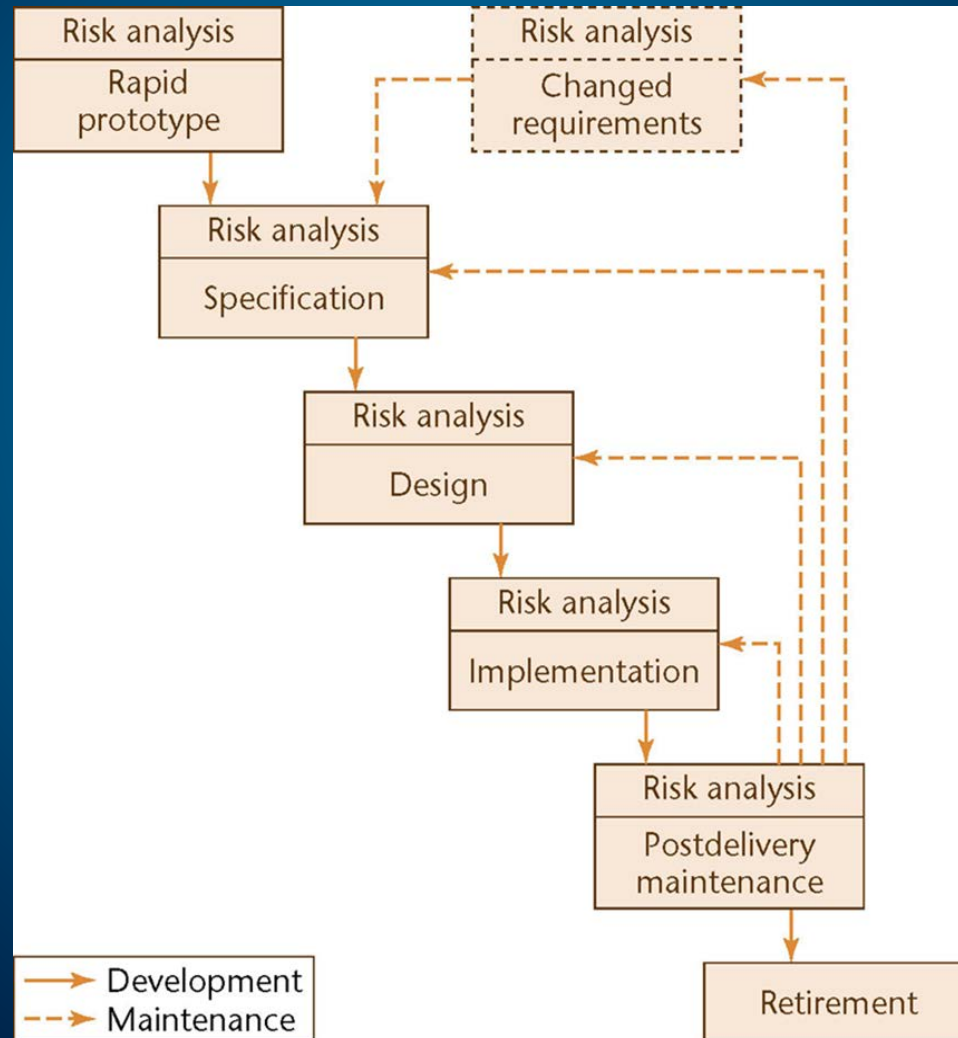
- Synchronize-and-Stabilize Life-Cycle Model

Synchronize-and-Stabilize Model

- Microsoft's life-cycle model
- Requirements analysis — interview potential customers
- Draw up specifications
- Divide project into 3 or 4 builds
- Each build is carried out by small teams working in parallel

Software Life-Cycle Models

- Spiral Life-Cycle Model



Comparison of Life-Cycle Models

Life-Cycle Model	Strengths	Weaknesses
Evolution-tree model (Section 2.2)	Closely models real-world software production Equivalent to the iterative-and-incremental model	
Iterative-and-incremental life-cycle model (Section 2.5)	Closely models real-world software production Underlies the Unified Process	
Code-and-fix life-cycle model (Section 2.9.1)	Fine for short programs that require no maintenance	Totally unsatisfactory for nontrivial programs
Waterfall life-cycle model (Section 2.9.2)	Disciplined approach Document driven	Delivered product may not meet client's needs
Rapid-prototyping life-cycle model (Section 2.9.3)	Ensures that the delivered product meets the client's needs	Not yet proven beyond all doubt
Open-source life-cycle model (Section 2.9.4)	Has worked extremely well in a small number of instances	Limited applicability Usually does not work
Agile processes (Section 2.9.5)	Work well when the client's requirements are vague	Appear to work on only small-scale projects
Synchronize-and-stabilize life-cycle model (Section 2.9.6)	Future users' needs are met Ensures that components can be successfully integrated	Has not been widely used other than at Microsoft
Spiral life-cycle model (Section 2.9.7)	Risk driven	Can be used for only large-scale, in-house products Developers have to be competent in risk analysis and risk resolution

UNIT 3: THE SOFTWARE PROCESS

(Chapter 3)

Learning Objectives: In this unit, you should be able to:

- Explain why two-dimensional life-cycle models are important.
- Describe the five core workflows of the Unified Process
- List the artifacts tested in the test workflow.
- Describe the four phases of the Unified Process.
- Explain the difference between the workflows and the phases in the unified process.
- Appreciate the importance of software process improvement.
- Describe the capability maturity model (CMM).

THE SOFTWARE PROCESS

- What is *Software Process*?
- The *software process* is simply the way we produce software. The way we produce software (*methodology*) will be different depending on the era the software is being developed (*Classical Paradigm & Object Oriented Paradigm methodology*).
- The *software process* methodology also incorporates the underlying *software life-cycle* model, techniques, tools, and mostly the individual building the software.

THE SOFTWARE PROCESS

- However, different individual or organization have different software development processes. This difference can be identified in:
 - Documentation: Some organizations consider the software they produce to be self-documenting; and other organizations, however, are documentation intensive.
 - Intensity of Testing: Some organizations devote up to half of their software budgets to testing the software, whereas others feel that only the user can thoroughly test the software product.
 - Post-delivery Maintenance: Most software organization continuously enhance their software to meet changing needs by releasing update/newer versions. Other organizations/individuals are busy with research, leaving development let alone maintenance to others.

THE SOFTWARE PROCESS

- Why does *software process* vary so drastically from organization to organization/individual to individual?
 - Lack of Software Engineering skills
 - Lack of technical knowledge in software development and maintenance
 - Management outlook: An organization may decide that it is better to deliver a product on time even if it is not adequately tested, others may decide to test the product thoroughly and deliver the product late.

THE SOFTWARE PROCESS

- However, regardless of the exact procedure, the ***software development process*** is structured around five workflows:
 - Requirement workflow
 - Analysis (*specification*) workflow
 - Design workflow
 - Implementation workflow
 - Testing

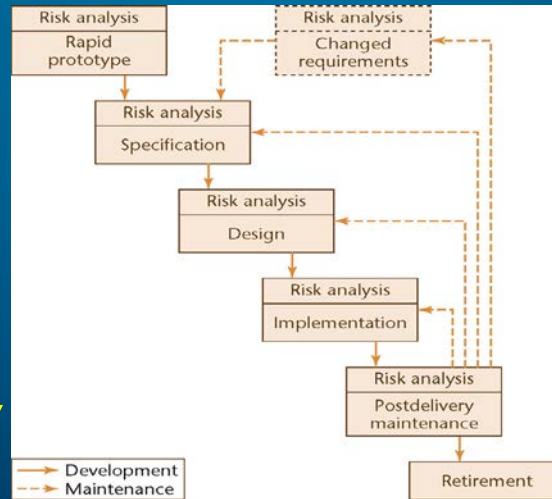
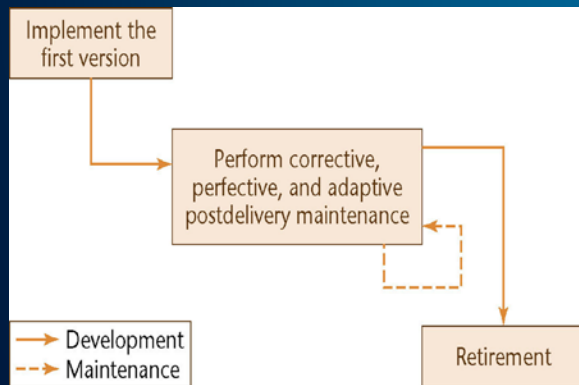
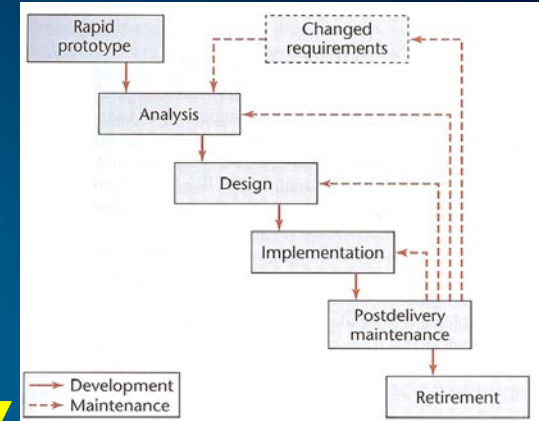
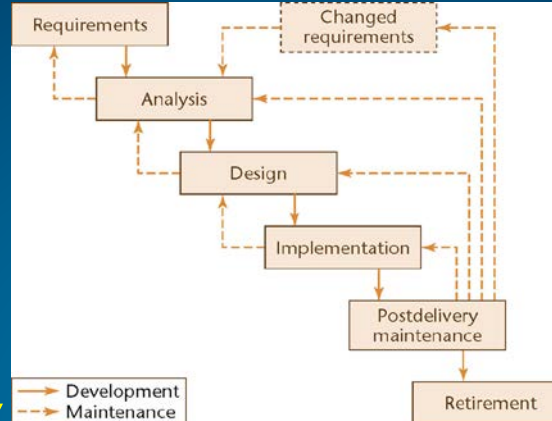
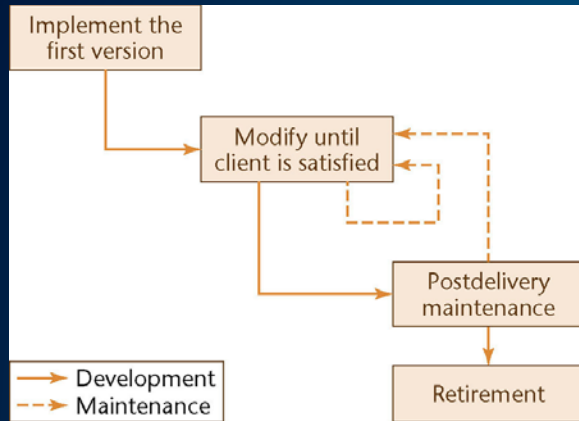
THE SOFTWARE PROCESS

1. Requirement workflow
2. Analysis (*specification*) workflow
3. Design workflow
4. Implementation workflow
5. Testing

THE UNIFIED PROCESS

- As stated earlier, *methodology* is one component of a software process e.g. classical and object-oriented methodology.
- The **Unified Process** is an object-oriented methodology primarily and mostly used nowadays for *object-oriented* software production.
- The **Unified Process** uses Unified Modeling Language (UML) to model the software developed.
- The **Unified Process** is not a specific series of steps that if followed will result in the construction of a software product.
- Instead, the **Unified Process** is an adaptable methodology, that can be modified for specific software product to be developed.

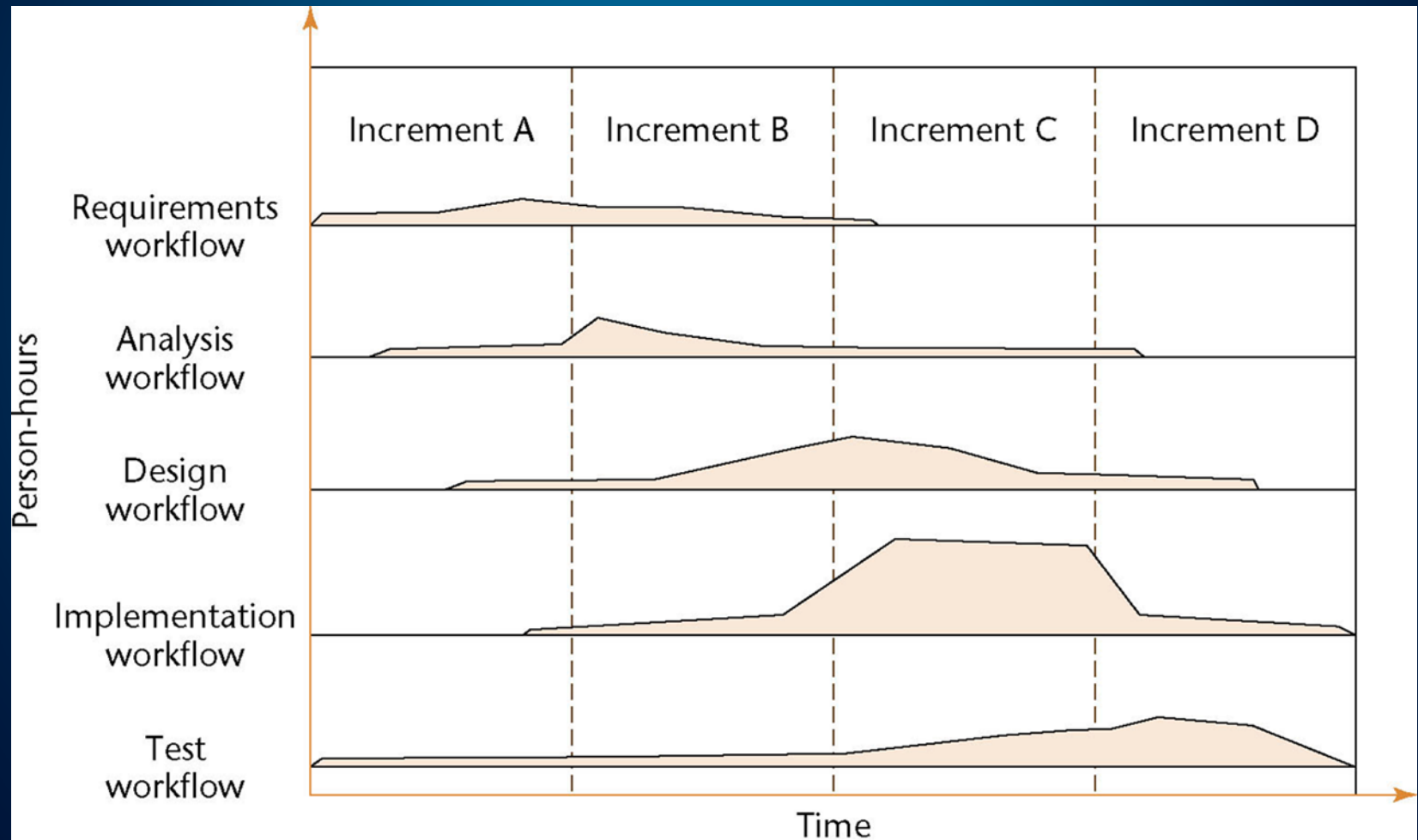
THE UNIFIED PROCESS



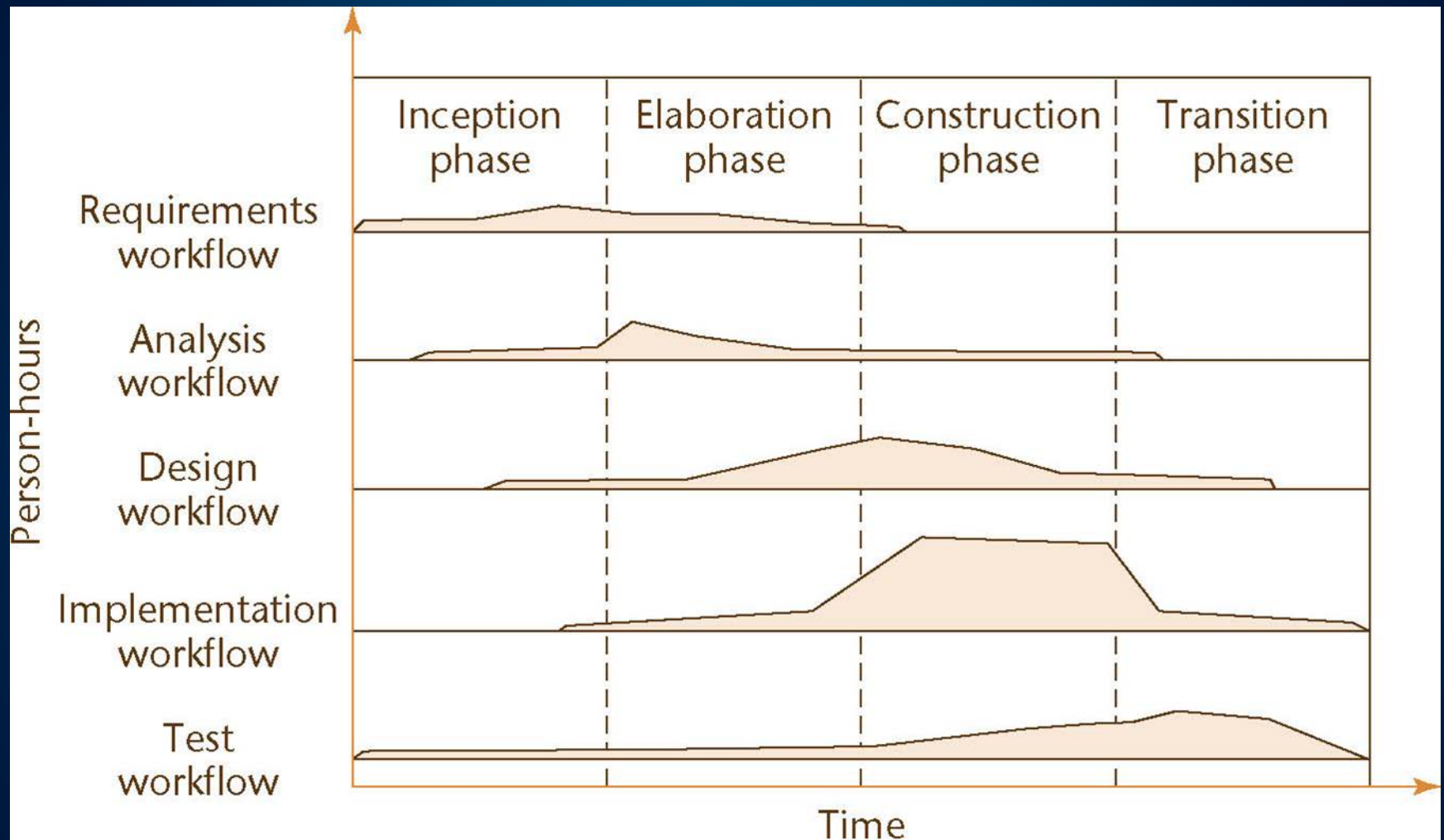
One Dimensional Life-Cycle Model

THE UNIFIED PROCESS

- Except:
 - Iteration and Incremental Model

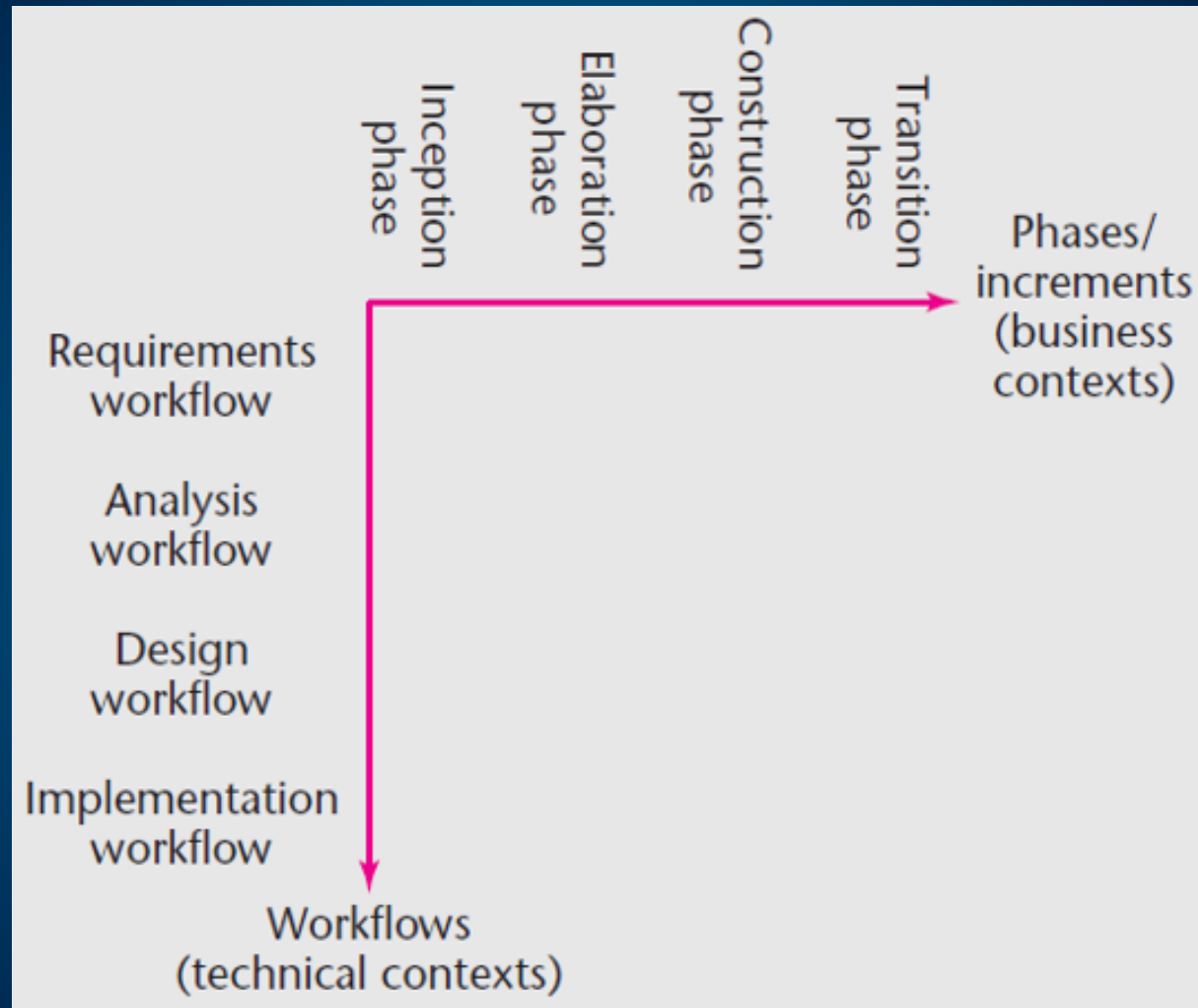


THE UNIFIED PROCESS



2-Dimensional Life Cycle Model

THE UNIFIED PROCESS



2-Dimensional Life Cycle Model

THE UNIFIED PROCESS

- Workflow (Technical context of a step)
 - Requirement workflow
 - Analysis (specification) workflow
 - Design workflow
 - Implementation workflow
 - Test workflow
 - Post-delivery maintenance
 - Retirement
- Phase (Business context of a step)
 - Inception phase
 - Elaboration phase
 - Construction phase
 - Transition phase

WORKFLOW OF THE UNIFIED PROCESS

The Requirement Workflow

- The aim of the requirements workflow is to determine the client's needs.
- The first task of the development team is to acquire basic understanding of the application domain the software is being developed for, by studying the business case.
- Meeting between the client and the developer to determine:
 - Deadline
 - Reliability
 - Cost
- The preliminary investigation of the clients need is called **Concept Exploration**.

WORKFLOW OF THE UNIFIED PROCESS

The Analysis Workflow

- The aim of the *analysis workflow* is to analyze and refine the requirements to achieve the detailed understanding of the requirements essential for developing a software product correctly.
- The key point is that the output of the *requirement workflow* must be totally comprehended by the client, clear of *ambiguities*.
- Once the specifications have been approved by the client, the preparation of the **SPMP** commences, showing the deliverables, milestones and budget.
- A software project management plan (SPMP) must be drawn up that reflects the separate workflows of the development process.

WORKFLOW OF THE UNIFIED PROCESS

The Design Workflow

- The specification of a product spell out *what* the product is to do; the design shoes *how* the product is to do it.
- The aim of the *design workflow* is to refine the components of the *analysis workflow* until the material is in a form that can be implemented by programmers.
- The designers decompose the product into **modules** (*class*) and once this is done a detailed design is performed.

WORKFLOW OF THE UNIFIED PROCESS

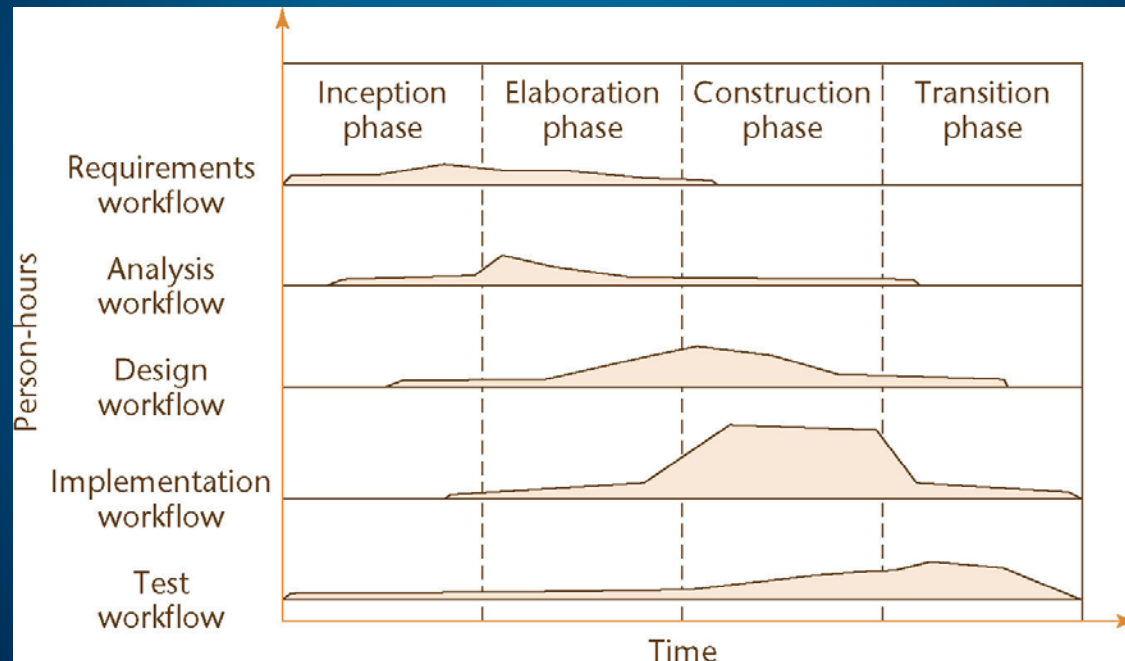
The Implementation Workflow

- The aim of the *implementation workflow* is to implement the designed target software product in the chosen implementation language(s).
- Software subsystems (*modules*) which consists of components or code artifacts are implemented by programmer.

WORKFLOW OF THE UNIFIED PROCESS

The Test Workflow

- In **Unified Process** testing is carried out in parallel with the other workflows starting from the beginning.



- The nature of the test workflow changes depending on the *artifacts* being tested.

WORKFLOW OF THE UNIFIED PROCESS

The Test Workflow

- Requirement Artifacts: If *requirement artifacts* is testable over the life cycle of the software product, then it will have **traceability** property. i.e. it will be possible to trace the *artifacts* from the *analysis workflow* back to the *requirement artifacts*.
- Analysis Artifacts: As stated earlier, the major source of faults in developed software is faults in specifications/SPMP that are not detected until the software has been developed and installed for the client. Therefore, both the analysis team and SQA group must check the analysis artifacts very well.

WORKFLOW OF THE UNIFIED PROCESS

The Test Workflow

- Design Artifacts: A critical aspect of testability is traceability. In case of design, every part of design must be linked to the analysis artifacts. Member of the design team and SQA group work through the design as a whole ensuring the design is correct.

WORKFLOW OF THE UNIFIED PROCESS

The Test Workflow

- Implementation Artifacts: Each components should be tested while it is being implemented, and after it has been implemented. The SQA performs the unit testing to test each software component. The integration testing is done to check that the components combine correctly to achieve the product specification. The SQA also performs the product testing, and the final step in testing the implementation artifacts is acceptance testing. In case of COTS, after product testing is completed, the first version termed the *alpha release*; the corrected alpha release is called the *beta release*.

WORKFLOW OF THE UNIFIED PROCESS

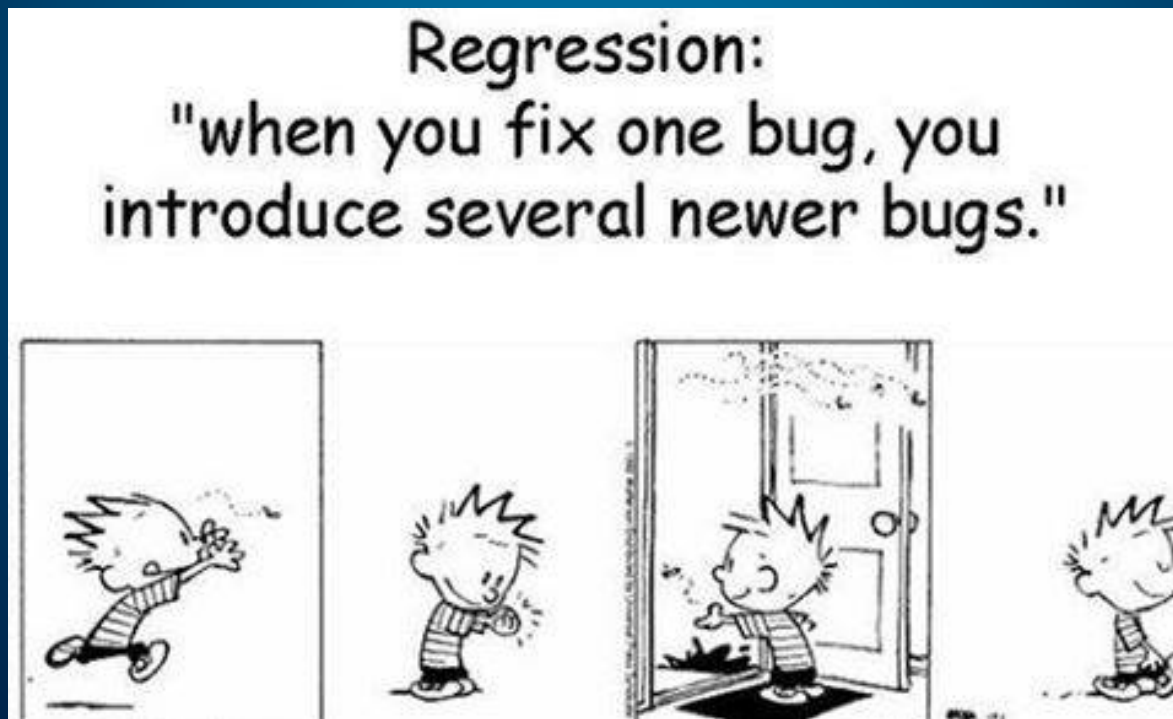
The Postdelivery Maintenance

- Postdelivery maintenance is an integral part of the software process that must be planned for from the beginning. Coding must be performed with future maintenance kept in mind.
- The common problem with Postdelivery maintenance is the lack of documentation.
- Effort must also be made prevent **regression fault** during maintenance

WORKFLOW OF THE UNIFIED PROCESS

The Postdelivery Maintenance

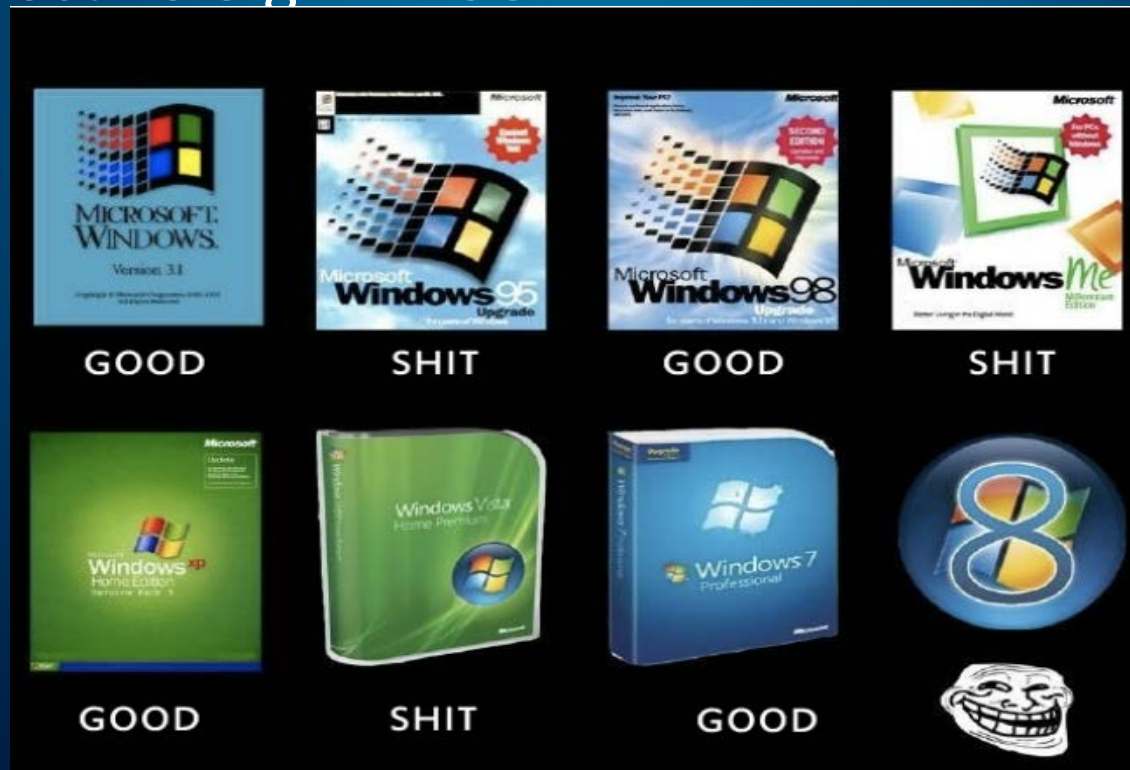
- Effort must also be made prevent regression fault during maintenance



WORKFLOW OF THE UNIFIED PROCESS

Retirement

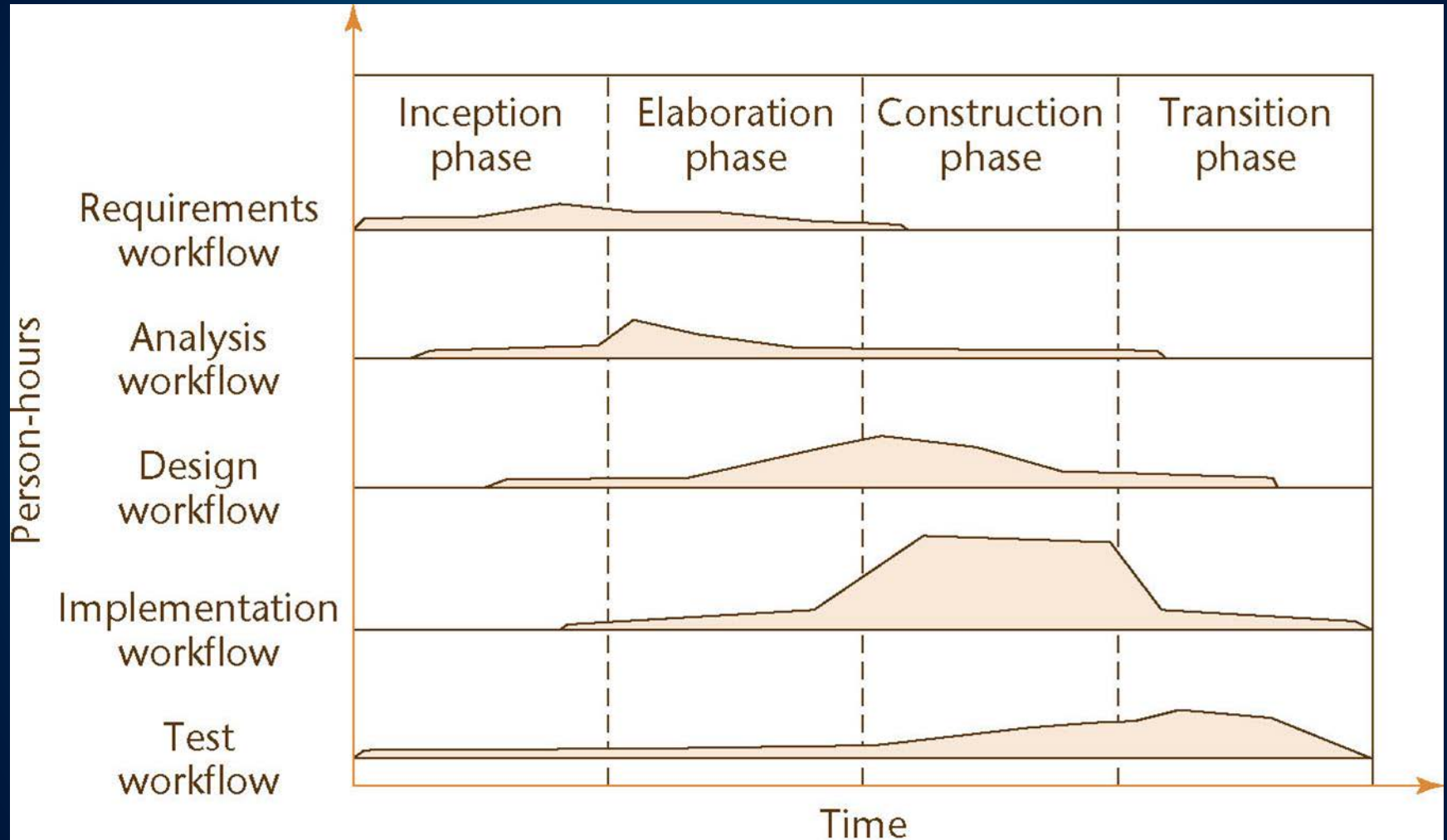
- The final stage in the software life cycle is retirement.
- After years of service, the software will reach a stage when further Postdelivery maintenance is no longer cost effective e.g. Win98



PHASES OF THE UNIFIED PROCESS

- The phases of the Unified Process replaces the incremental stages. The phases are:
 - Inception phase
 - Elaboration phase
 - Construction phase
 - Transition phase
- Every step performed in the **Unified Process** falls into one of five core *workflows* and also one or four *phases*.
- The combination of the *workflow* and *phases* makes the **Unified Process** a two-dimensional life-cycle model.

PHASES OF THE UNIFIED PROCESS



PHASES OF THE UNIFIED PROCESS

The Inception Phase

- The aim of the inception phase (*first increment*) is to determine whether is worthwhile to develop the target software product.
- The inception phase concentrate mostly on the requirement workflow of the software development by obtaining the business model and domain knowledge of the target domain.
- The next step after the inception phase is to identify the risks
 - Technical risks
 - Not getting the requirements right
 - Not getting the architecture right

PHASES OF THE UNIFIED PROCESS

The Inception Phase

- At this phase the following questions needs to be answered:
 - Is the proposed software product cost effective?
 - Can the proposed software product delivered on time?
 - What risks are involved in developing the software product, and how can these risks be mitigated?
- Also understanding the risk:
 - Technical risk
 - No getting the requirements correctly
 - No getting the architecture right

PHASES OF THE UNIFIED PROCESS

The Inception Phase

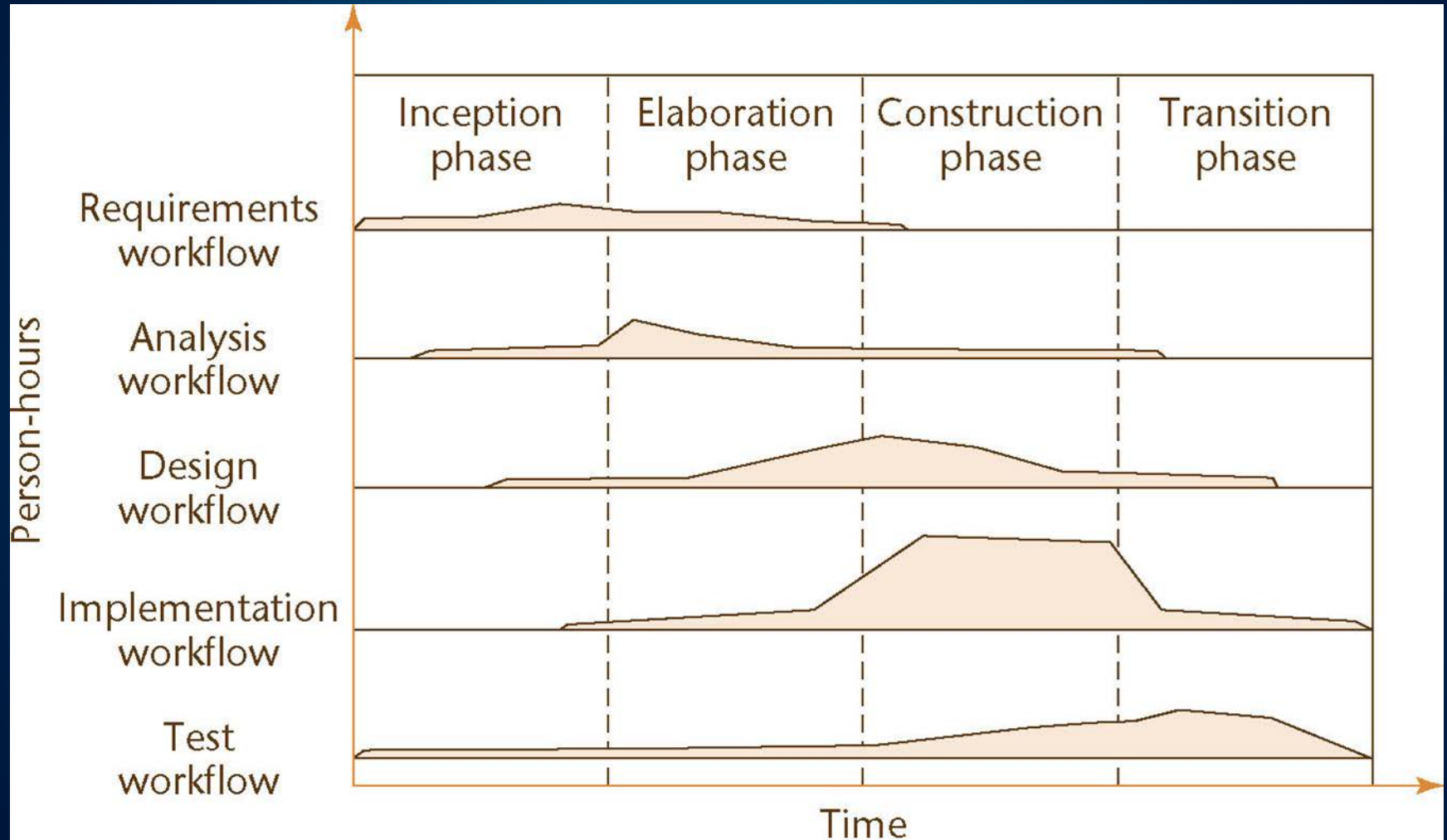
- The delivery of the inception phase includes:
 - The initial version of the domain model
 - The initial version of the business model
 - The initial version of the requirement artifacts
 - A preliminary version of the analysis artifacts
 - A preliminary version of the architecture
 - The initial list of risks
 - The initial uses cases
 - The initial version of the business case

PHASES OF THE UNIFIED PROCESS

The Elaboration Phase

- The aim of the elaboration phase (*second increment*) is to refine the requirements, refine the architecture, monitor the risks and refine their priorities, refine the business case and produce the **SPMP**.
- The deliverables of the elaboration phase include:
 - The completed domain model
 - The completed business model
 - The completed requirements artifacts
 - The completed analysis artifacts
 - An updated version of the architecture
 - An updated list of risks
 - The SPMP
 - The completed business case

PHASES OF THE UNIFIED PROCESS

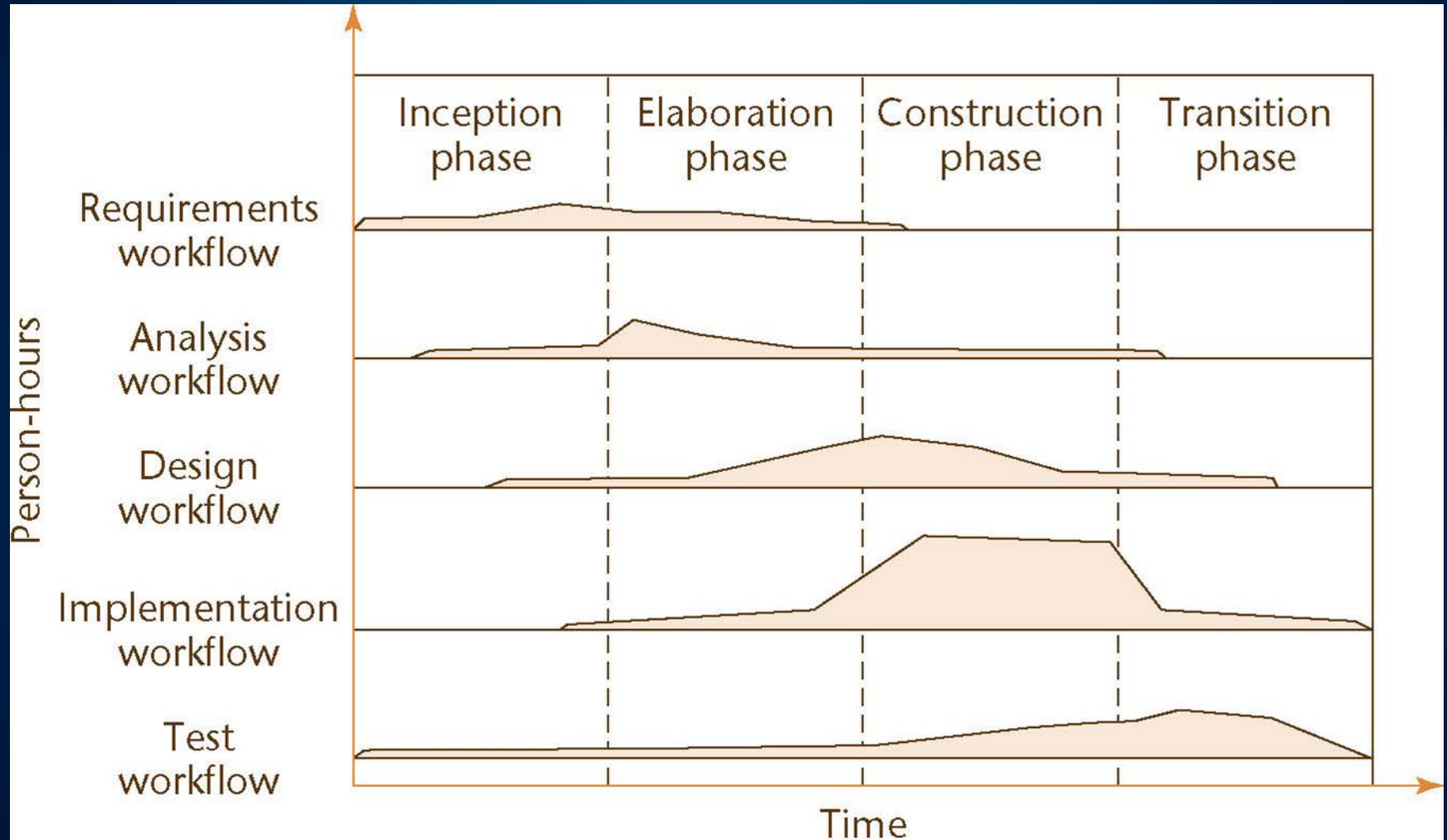


PHASES OF THE UNIFIED PROCESS

The Construction Phase

- The aim of the construction phase (*third increment*) is to produce the first operational quality version of the software product the *beta* release.
- The deliverables of the construction phase include:
 - The initial user manual and other manual
 - All the artifacts
 - The completed architecture
 - The updated risk list
 - The SPMP
 - Updated business case

PHASES OF THE UNIFIED PROCESS

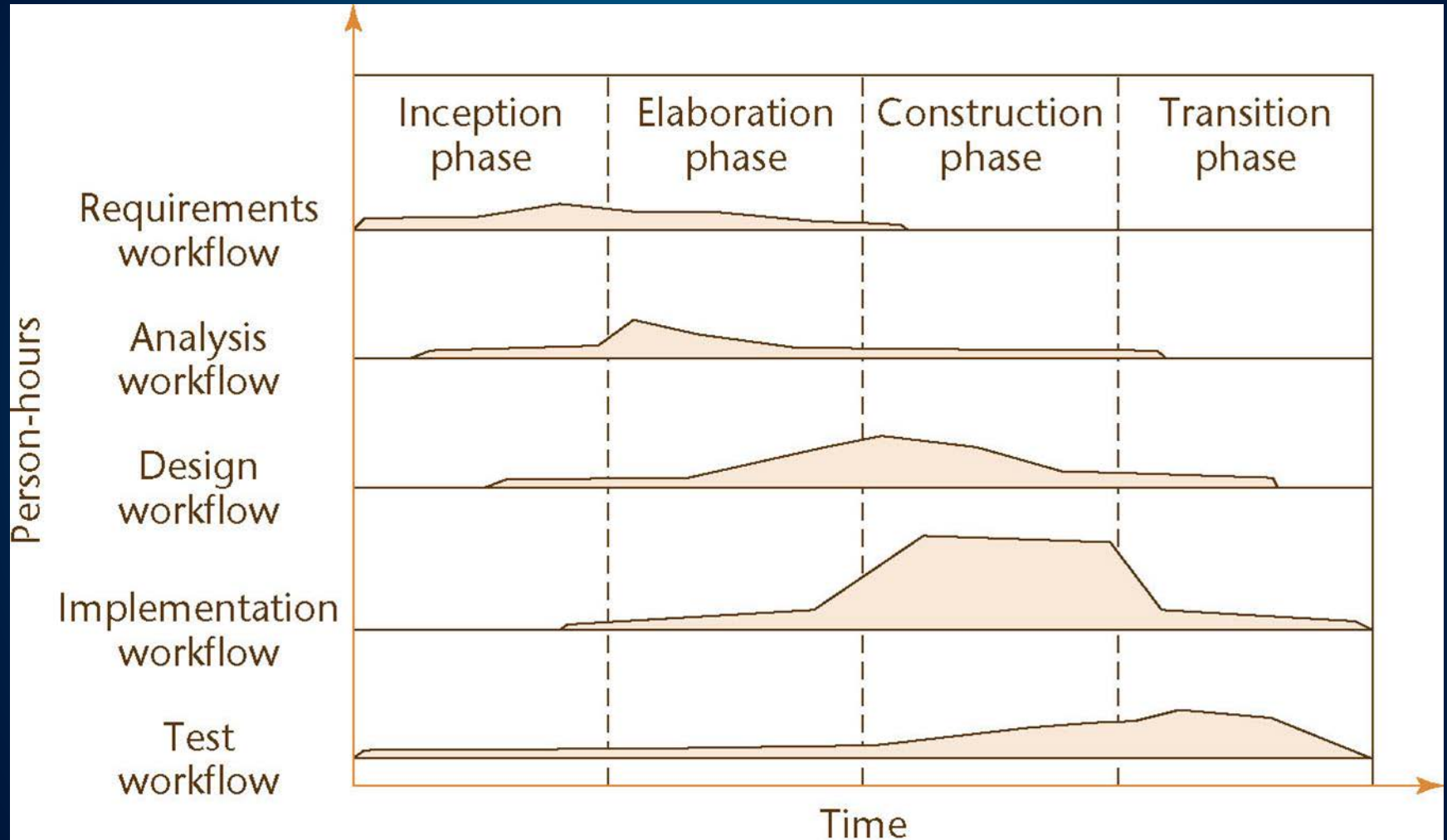


PHASES OF THE UNIFIED PROCESS

The Transition Phase

- The aim of the transition phase (*fourth increment*) is to ensure that the client's requirements have indeed been met.
- The deliverables of the construction phase include:
 - All the artifacts
 - The completed manuals.

PHASES OF THE UNIFIED PROCESS



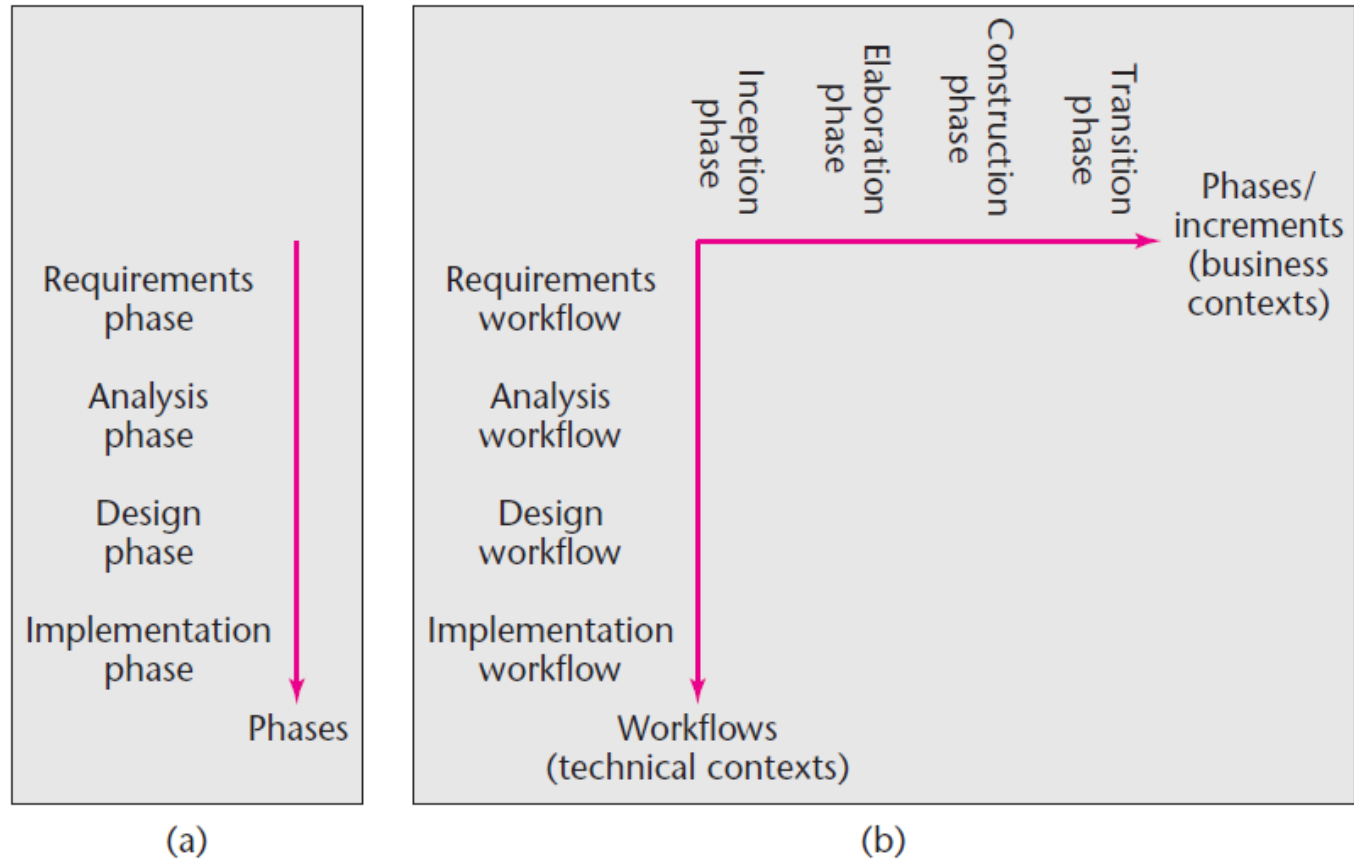
ONE- VERSUS TWO-DIMENSIONAL LIFE-CYCLE MODELS

- ???
- Classical life-cycle model e.g. waterfall model is a *one-dimensional model* as represented by the single axis.
- The **Unified Process** is a two-dimensional *life-cycle model*.
- The two-dimensional model is much better than the one-dimensional model because software products are too large to handle as a single unit. Instead, the task has to be divided into phases according to the workflows.

ONE- VERSUS TWO-DIMENSIONAL LIFE-CYCLE MODELS

FIGURE 3.2

Comparison of
(a) a classical
one-dimensional
life-cycle model
and (b) the two-
dimensional
Unified Process
life-cycle
model.



IMPROVING THE SOFTWARE PROCESS

- Improving software development process is very important in order to produce quality software. Efforts have been made towards this, with some institutions founded to ensure the improvement in software process. Also with standard enacted e.g.
 - US DoD founded Software Engineering Institute (SEI) in 1987 e.g. Capability Maturity Models (CMM)
 - International Organization for Standardization (ISO) e.g. ISO-900: ISO/IEC 15504.
 - Licensing Software Engineers - IEEE Computer Society Certified Software Development Professional

IMPROVING THE SOFTWARE PROCESS

Professional Software Engineering Master Certification

This professional competency certification requires successful completion of Certificates of Proficiency in the following eleven key knowledge areas: Software Requirements, Software Design, Software Construction, Software Testing, Software Maintenance, Software Configuration Management, Software Quality, Software Engineering Processes, Software Engineering Models & Methods, Software Engineering Management, Software Engineering Economics and the successful completion of two advanced applied modules.

Recommended Prerequisites:

Candidates seeking this professional certification should have completed a minimum of four years of college education in computer science or equivalent in a related field and a minimum of four years of relevant industry experience. However, there are no prerequisites required for registration.

Requirements:

This professional competency certification requires the successful completion of the following 2 parts.

Part I - IEEE CS Exam. One online exam (containing questions from all 11 knowledge areas):

1. Software Requirements
2. Software Design
3. Software Construction
4. Software Testing
5. Software Maintenance
6. Software Configuration Management
7. Software Quality
8. Software Engineering Process
9. Software Engineering Models & Methods
10. Software Engineering Management
11. Software Engineering Economics

Part II - PROXOR. The successful completion of:

- Applied Exam Module III
- Applied Exam Module IV

IMPROVING THE SOFTWARE PROCESS

Part I – IEEE CS Exam

PSEM exam contains the following knowledge areas:

- Software Requirements
- Software Design
- Software Construction
- Software Testing
- Software Maintenance
- Software Configuration Management
- Software Quality
- Software Engineering Process
- Software Engineering Models & Methods
- Software Engineering Management
- Software Engineering Economics



Part II PROXOR Applied Modules (III and IV)



**Professional
Software
Engineering
Master
Certification
(PSEM)**

Part I - IEEE CS Exam Duration:	180 minutes each
Exam Questions:	160 questions (one online exam from the KAs listed)
Exam Location:	Exams are 100% online and proctored remotely
Part II - PROXOR Exam Duration:	Applied Module III - 180 minutes Applied Module IV - 180 minutes

Professional Software Engineering Master Bundle

Non-Members: \$1200
Members: \$890
Student Members: \$445

Professional Software Engineering Master Exams

Non-Members: \$865
Members: \$690

Recertification:



Please [download](#) the recertification form.

IMPROVING THE SOFTWARE PROCESS

Essential Information

Computer software developers apply mathematic and scientific principles to the design and development of software systems and applications. The minimum educational requirement is a bachelor's degree, and certification may be necessary.

Required Education	Bachelor's in computer science or software engineering
Other Requirements	Certification may be needed
Projected Job Growth (2014-2024)*	19% for applications software developers; 13% for systems software developers
Median Salary (May 2015)*	\$98,260 annually for applications software developers; \$105,570 annually for systems software developers

Source: *U.S. Bureau of Labor Statistics.

Certifications

Software developers and engineers do not have professional licensing requirements but may need certifications. However, if one chose to certify, one possible option is certification from the Computer Society of the Institute of Electrical and Electronics Engineers, which offers software development credentials for beginning and experienced software engineers.

CAPABILITY MATURITY MODELS

- The capability maturity models (CMM) of the SEI are group of strategies for improving the software process, irrespective of the actual life-cycle model used.
- The SEI has developed CMM for software called **SW-CMM**.
- The strategy of the SW-CMM is to improve the management of the software process incrementally.
- The **SW-CMM** has five level of maturity

CAPABILITY MATURITY MODELS

- **Maturity Level 1 - Initial Level:** At the initial level, essentially no sound software engineering management practices are in place in the organization. Everything is done on an ad hoc basis.
- **Maturity Level 2 - Repeatable Level:** At the repeatable level, basic software project management practices are in place.
- **Maturity Level 3 - Defined Level:** At the defined level, the process for software production is fully documented. Both the managerial and technical aspects of the process are clearly defined and efforts are made to improve the process.

CAPABILITY MATURITY MODELS

- **Maturity Level 4 - Managed Level:** A managed-level organization sets quality and productivity goals for each project. These qualities are measured continually and corrective action is taken when unacceptable deviations from the goal is noticed.
- **Maturity Level 5 - Optimizing Level:** The goal of the optimized-level organization is continuous process improvement. Statistical quality and process control techniques are used to guide the organization.

CAPABILITY MATURITY MODELS

5. Optimizing level: Process control	Defect prevention Technology change management Process change management
4. Managed level: Process measurement	Quantitative process management Software quality management
3. Defined level: Process definition	Organization process focus Organization process definition Training program Integrated software management Software project engineering Intergroup coordination Peer reviews
2. Repeatable level: Basic project management	Requirements management Software project planning Software project tracking and oversight Software subcontract management Software quality assurance Software configuration management
1. Initial level: Ad hoc process	Not applicable

CAPABILITY MATURITY MODELS

- The US Air Force stipulated that any software development organization that wished to be an Air Force contractor had to conform to **SW-CMM level 3**.

OTHER SOFTWARE PROCESS IMPROVEMENT INITIATIVES

- A different attempt to improve software quality is based on the **ISO 9000-series standards**, a series of five standard applicable to industrial activities, design, development, production, installation and servicing.
- ISO/IEC 15504 is an international process improvement initiative, like the ISO 9000.

COSTS AND BENEFITS OF SOFTWARE PROCESS IMPROVEMENT

- Does implementing software process improvement lead to increased profitability?
- Results indicate that this indeed is the case.
 - For example, the Software Engineering Division of Hughes Aircraft in California, spent nearly \$500,000 between 1987 and 1990 for assessments and improvement programs.
 - During this 3-year period, Hughes Aircraft moved from maturity level 2 to level 3, with expectation of future improvement to level 4 and even level 5.
 - As a consequence of improving its process, Hughes Aircraft estimated its annual savings to be the order of \$2 million.
 - These savings accrued in a number of ways, including decreased overtime hours, fewer crises, improved employee morale etc.

SUMMARY

- Software Process
- Unified Process
- Categorization of Unified Process; Workflow and Phases
- The importance of two-dimensional life-cycle
- Software process improvement
- Capability Maturity Model
- Cost effectiveness of software process improvement.

UNIT THREE COMPLETED

CLASS TEST

4th October 2021